ADMINISTRACIÓN DE SERVICIOS DE SISTEMAS OPERATIVOS

(88020)

Práctica 2

Sistema de archivos, comandos, usuarios y permisos

Proyecto AURORA-6 · AURORA Aerospace

Grupo 3

Guillermo Romo Sánchez

Alonso Malvar Ariza

Ariadna Sánchez Villa

Docente: Domingo Sampedro Lirio

Grado en Inteligencia Artificial y Robótica

Curso 2026-2027 · Octubre de 2026

ÍNDICE

1. Introducción, objetivos y entorno de trabajo 3
1.1. Introducción 3
1.2. Objetivos 3
1.3. Roles del grupo 3
1.4. Entorno y topología 4
2. Primer acceso al sistema e identificación básica 6
2.1. Por qué no se usa root como cuenta habitual 9
3. Creación de la estructura de la empresa 10
4. Usuarios, grupos y roles 12
4.1. Creación de grupos y cuentas 12
4.2. Pertenencia a los grupos de departamento 12
4.3. Comprobación 12
5. Permisos tradicionales (DAC) y configuración 751 14
5.1. Interpretación de 7, 5 y 1 14
6. Política de acceso por roles y ACLs 16
6.1. Traducción de la política 16
6.2. Criterios de diseño 17
6.3. Cuándo es necesaria una ACL y qué aporta 17
6.4. Configuración realizada por cada integrante 18
6.5. Archivos de ejemplo para las pruebas 20
7. Validación individual de la política de permisos 22
7.1. Pruebas del miembro A: Guillermo (usuario_a) 22
7.2. Pruebas del miembro B: Alonso (alonso) 24
7.3. Pruebas del miembro C: Ariadna (ariadna) 25
7.4. Interpretación 27
8. Incidencias y soluciones 28
9. Conclusiones 30
10. Bibliografía 31
Anexo A. Script de configuración del servidor 32
Anexo B. Preguntas previsibles en la defensa 35

1. Introducción, objetivos y entorno de trabajo

1.1. Introducción

En esta práctica administramos el servidor Ubuntu Server instalado en la P1 utilizando exclusivamente la terminal. Sobre él preparamos la infraestructura de la empresa ficticia AURORA Aerospace (proyecto AURORA-6): una estructura de directorios en `/srv/aurora6`, una cuenta Linux por integrante, los grupos de cada departamento y una política de acceso que combina permisos tradicionales (DAC) con listas de control de acceso (ACL). La misma infraestructura se reutilizará en las prácticas P3, P4 y P5, por lo que las decisiones que se toman aquí se han documentado de forma que se puedan reproducir.

A diferencia de la P1, el trabajo se organiza como un sistema real compartido: un ordenador del grupo actúa como servidor (ejecuta la máquina virtual) y los otros dos ordenadores actúan como clientes desde los que cada integrante inicia sesión con su propia cuenta. De esta forma cada miembro aporta evidencias de operaciones permitidas y denegadas con su identidad, tal como exige la regla de trabajo individual del enunciado.

1.2. Objetivos

- Identificar el usuario, el nombre del sistema y la jerarquía básica de directorios del servidor.
- Crear desde la terminal la estructura de directorios de la empresa en `/srv/aurora6`.
- Crear las cuentas de los tres integrantes, los grupos de departamento y el grupo común `proyecto`.
- Interpretar los permisos tradicionales a partir de la configuración 751.
- Traducir la política empresarial de roles a propietarios, grupos, permisos DAC y ACLs, justificando cuándo es necesaria una ACL.
- Validar la política con pruebas reales de cada usuario: operaciones permitidas y denegadas.

1.3. Roles del grupo

Miembro	Integrante	Cuenta Linux	Rol	Grupos
A	Guillermo Romo Sánchez	`usuario_a`	Ingeniería mecánica	mecanica, proyecto
B	Alonso Malvar Ariza	`alonso`	Aviónica y software	avionica, software, proyecto
C	Ariadna Sánchez Villa	`ariadna`	Compras y configuración	compras, proyecto

Tabla 1. Reparto de roles y cuentas de trabajo

El enunciado permite usar nombres reales o nombres acordados equivalentes. Alonso y Ariadna usan su nombre como cuenta Linux (alonso y ariadna). Para el miembro A se usa usuario_a, porque el nombre guillermo ya corresponde a la cuenta de administración creada en la P1, que pertenece al grupo sudo, y las cuentas de trabajo no deben tenerlo. Las cuentas de B y C se crearon primero como usuario_b y usuario_c y después se renombraron (sección 8). En el script del Anexo A basta con cambiar las tres variables del principio.

1.4. Entorno y topología

Elemento	Valor
Hipervisor	Oracle VirtualBox (PC servidor del grupo)
Máquina virtual	server-asio-g3, Ubuntu Server 26.04.1 LTS (kernel 7.0.0-38) instalado en la P1
Red de la VM	NAT con reenvío de puertos (TCP anfitrión 2222 hacia invitado 22)
Clientes	Los otros dos ordenadores del grupo, con el cliente OpenSSH (`ssh`)
Cuenta de administración	guillermo, creada en la P1 (miembro del grupo sudo)
Cuentas de trabajo	`usuario_a`, `alonso`, `ariadna` (sin privilegios `sudo`)
Paquetes adicionales	`acl` (`setfacl`, `getfacl`) y `tree`

Tabla 2. Entorno de la práctica

Figura 1. Esquema de acceso al servidor (las IP entre corchetes dependen de la red del aula)

```
PC cliente 1 --+
               +--> [IP_PC_servidor]:2222 (PC servidor, Windows + VirtualBox)
PC cliente 2 --+         |
                         | NAT, reenvío de puertos 2222 -> 22
                         v
               VM server-asio-g3, 10.0.2.15:22 (sshd)
               (también accesible desde la consola de la VM)
```

La VM sigue en modo NAT, como se justificó en la P1, porque garantiza acceso a Internet sin depender de la red del aula. La limitación del NAT es que la VM no es accesible desde fuera del anfitrión salvo a través de las reglas de reenvío. En la P1 la regla de SSH escuchaba en `127.0.0.1`, de modo que solo admitía conexiones desde el propio PC servidor. Para que los otros dos ordenadores puedan conectarse hay que hacer que la regla escuche en la interfaz de red del anfitrión y permitir ese puerto en el firewall de Windows.

Configuración del PC servidor

1. En VirtualBox: Configuración de `server-asio-g3` > Red > Adaptador 1 (NAT) > Avanzadas > Reenvío de puertos. En la regla `ssh` se deja vacío el campo IP anfitrión (escucha en todas las interfaces del anfitrión), puerto anfitrión 2222, IP invitado `10.0.2.15` y puerto invitado 22. La IP del invitado se comprueba dentro de la VM con `ip -4 addr`.
2. Se averigua la IP del PC servidor en la red local con `ipconfig` (campo Dirección IPv4 del adaptador Wi-Fi o Ethernet).
3. Se abre el puerto 2222 en el firewall de Windows solo para la subred local, desde PowerShell ejecutado como administrador:

```
New-NetFirewallRule -DisplayName "ASSIO SSH VM 2222" -Direction Inbound -Protocol TCP -
LocalPort 2222 -RemoteAddress LocalSubnet -Action Allow -Profile Any
```

`New-NetFirewallRule` crea una regla de entrada (`-Direction Inbound`) para TCP 2222. `-RemoteAddress LocalSubnet` limita la regla a equipos de la misma subred, de modo que no se expone el puerto a cualquier origen. Se usa `-Profile Any` porque las redes de la universidad suelen clasificarse

como públicas. Al terminar la sesión de prácticas la regla se elimina con `Remove-NetFirewallRule -DisplayName "ASSIO SSH VM 2222"`.

Comprobación en la VM y conexión desde los clientes

En la VM (cuenta de administración)

```
sudo systemctl status ssh.socket ssh      # desde Ubuntu 24.04 SSH se activa por socket
sudo ss -tlnp | grep ':22'                 # debe haber un socket escuchando en el puerto
22
sudo sshd -T | grep -i passwordauthentication   # debe ser "yes" para entrar con contraseña
```

En cada PC cliente

```
Test-NetConnection [IP_PC_servidor] -Port 2222     # PowerShell: TcpTestSucceeded : True
ssh -p 2222 usuario_a@[IP_PC_servidor]             # cada integrante con su propia cuenta
```

La primera conexión pide aceptar la huella del servidor, lo cual es normal. Si la red del aula aísla los equipos entre sí (situación habitual en redes Wi-Fi como eduroam), la prueba con `Test-NetConnection` falla aunque todo esté bien configurado. En ese caso los tres ordenadores pueden conectarse al punto de acceso de un móvil, o cada integrante puede iniciar sesión con su cuenta en la consola de la VM desde el PC servidor. La práctica no depende del medio de acceso: lo importante es que cada operación se ejecute con la cuenta del integrante correspondiente.

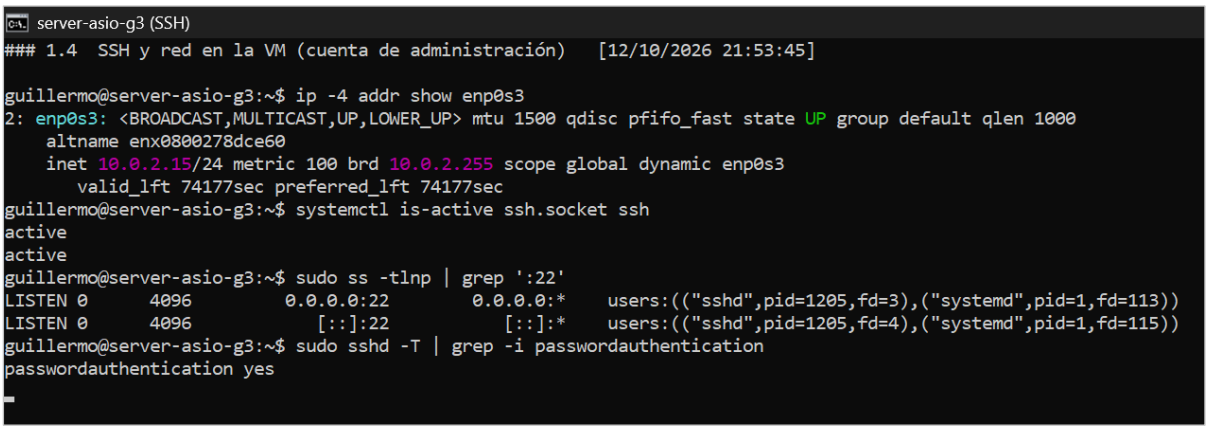


Figura 2. Comprobación de SSH y de la red dentro de la VM (cuenta guillermo)

Pendiente: captura de la regla de reenvío de VirtualBox con la IP de anfitrión vacía y de Test-NetConnection / ssh desde los PC de Alonso y Ariadna.

2. Primer acceso al sistema e identificación básica

Cada integrante ejecuta la misma secuencia de comandos en el servidor: Guillermo con la cuenta de administración (guillermo) y Alonso y Ariadna con su propia cuenta. El objetivo es comprobar quién soy dentro del sistema, en qué máquina estoy y cómo se organiza la jerarquía de directorios antes de modificar nada. Las salidas de Alonso y Ariadna se obtuvieron en el servidor ejecutando cada orden con su identidad (sudo -u alonso y sudo -u ariadna), porque en ese momento la regla de reenvío de puertos todavía escuchaba solo en 127.0.0.1 y no podían conectarse desde sus PC.

Comandos (cuenta de administración)

```
whoami
hostname
hostnamectl
pwd
ls -l /
ls -la /home
ls -la /root           # sin sudo: debe dar Permission denied
sudo ls -la /root      # con sudo: muestra el contenido
```

Comando	Qué muestra	Resultado esperado
`whoami`	Nombre del usuario efectivo de la sesión	El nombre de la cuenta de administración
`hostname` / `hostnamectl`	Nombre del equipo; `hostnamectl` añade sistema operativo, kernel, arquitectura y virtualización	`server-asio-g3`, Ubuntu, virtualización `oracle`
`pwd`	Directorio de trabajo actual	`/home/<admin>` tras iniciar sesión
`ls -l /`	Directorios de primer nivel del FHS	`bin`, `boot`, `etc`, `home`, `root`, `srv`, `usr`, `var`...
`ls -la /home`	Directorios personales de los usuarios normales	Una entrada por usuario con directorio personal
`ls -la /root`	Directorio personal de root	Sin `sudo`: Permission denied; con `sudo`: su contenido

Tabla 3. Comandos de identificación y resultado esperado

En la jerarquía estándar, `/` es la raíz de todo el sistema de archivos, `/home` contiene los directorios personales de los usuarios normales y `/root` es el directorio personal del superusuario. `/root` está fuera de `/home` y tiene permisos 700, por eso un usuario normal no puede listarlo y solo se puede consultar con `sudo`. El directorio `/srv`, que usaremos en el resto de la práctica, está reservado por el FHS para los datos que sirve el sistema, lo que lo hace adecuado para los datos compartidos de la empresa.

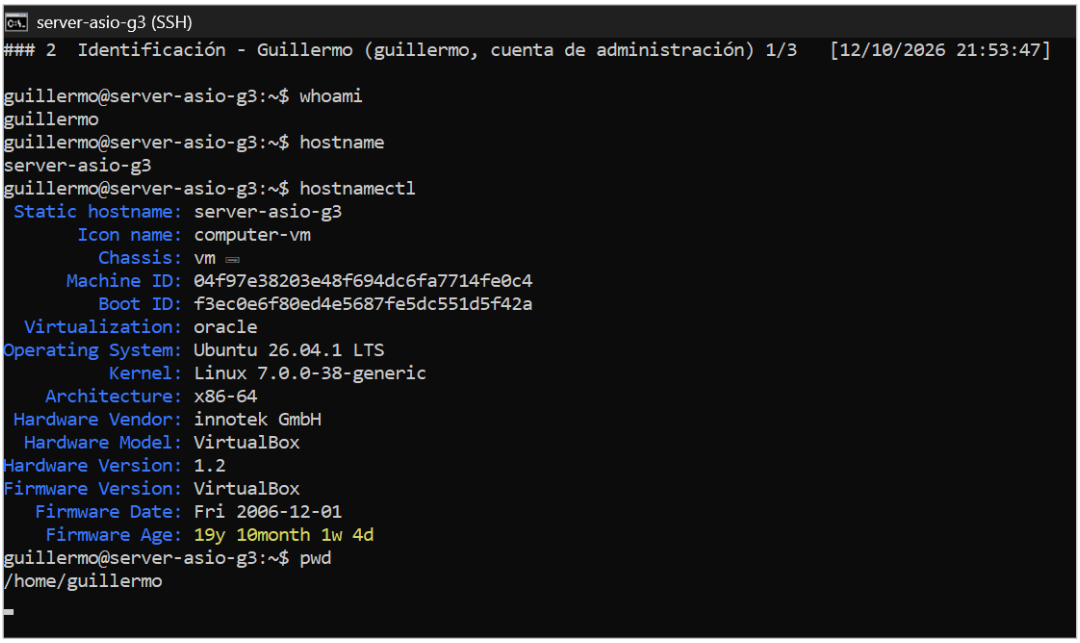


Figura 3. Identificación con la cuenta de administración guillermo (1/3)

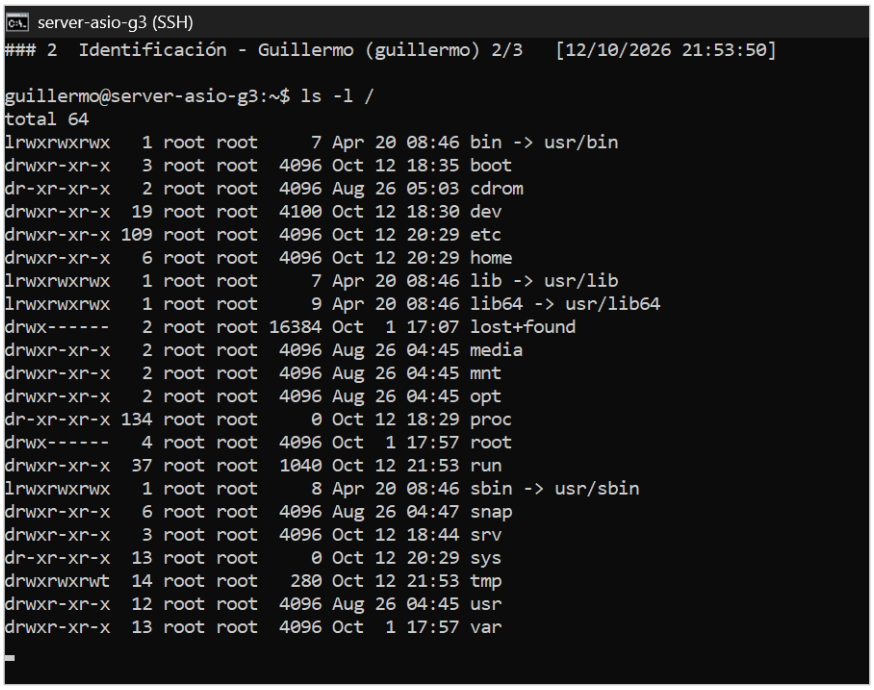


Figura 4. Jerarquía de primer nivel con ls -l / (2/3)

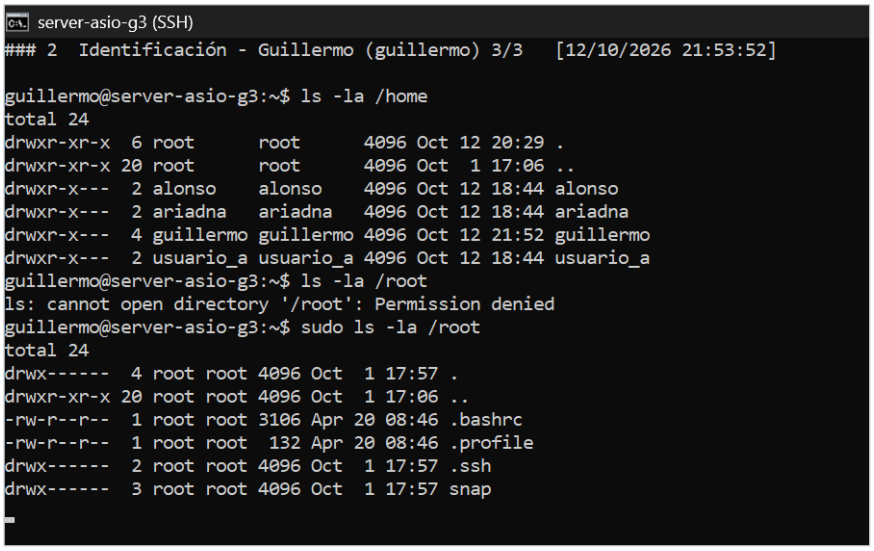


Figura 5. /home y /root sin y con sudo (3/3)

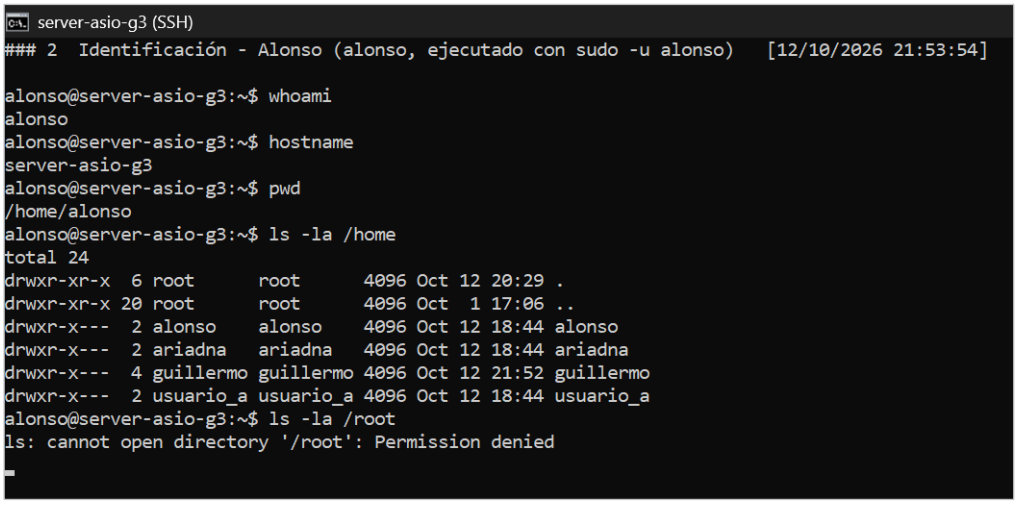


Figura 6. Identificación con la cuenta de Alonso (alonso)

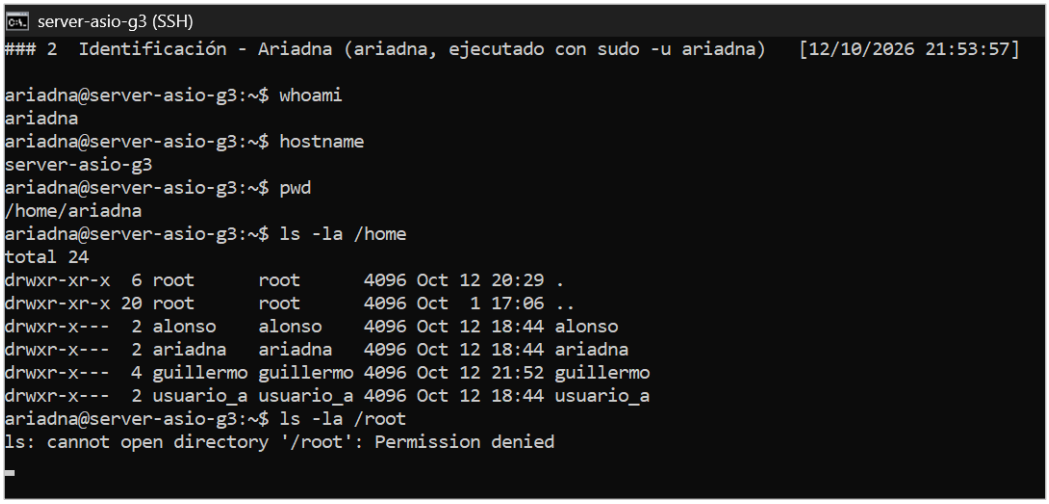


Figura 7. Identificación con la cuenta de Ariadna (ariadna)

2.1. Por qué no se usa root como cuenta habitual

La cuenta root no tiene ninguna restricción: el núcleo no le aplica comprobaciones de permisos sobre archivos, por lo que cualquier error de escritura (por ejemplo, un `rm -r` sobre la ruta equivocada o un `chmod -R` mal dirigido) se ejecuta sin confirmación y puede dejar el sistema inutilizable. Además, cualquier programa lanzado como root hereda todos sus privilegios, de modo que una vulnerabilidad en ese programa compromete el sistema completo.

Trabajar con una cuenta normal y elevar privilegios con `sudo` solo cuando hace falta aplica el principio de mínimo privilegio. `sudo` pide la contraseña del propio usuario, solo funciona para cuentas autorizadas en `/etc/sudoers` (en Ubuntu, el grupo `sudo`) y registra cada orden en el registro de autenticación (`/var/log/auth.log` o `journalctl`), lo que permite saber quién hizo cada cambio. En Ubuntu la cuenta root viene bloqueada por defecto precisamente por estas razones.

3. Creación de la estructura de la empresa

La estructura se crea con un solo `mkdir`. La opción `-p` crea los directorios intermedios que no existen y no da error si alguno ya existe, lo que hace el comando repetible. Las llaves `{a,b,c}` son expansión de llaves de Bash: el intérprete genera una ruta por cada elemento antes de ejecutar `mkdir`, de modo que no hace falta escribir cada ruta completa. Como `/srv` pertenece a root, la creación necesita `sudo`.

Creación de la estructura

```
sudo mkdir -p /srv/aurora6/ingenieria/{mecanica,avionica,software} \
              /srv/aurora6/compras/{proveedores,pedidos} \
              /srv/aurora6/contabilidad/facturas \
              /srv/aurora6/documentacion/comun \
              /srv/aurora6/confidencial \
              /srv/aurora6/intercambio/{incoming,approved,quarantine}
```

Comprobación

```
sudo apt update && sudo apt install -y tree acl
tree -d /srv/aurora6
find /srv/aurora6 -type d | sort          # alternativa sin instalar tree
```

tree -d muestra solo directorios en forma de árbol y su última línea indica el número total. En nuestro servidor la salida es «17 directories»: esta versión de tree cuenta también el propio /srv/aurora6, al que se suman 6 directorios de primer nivel y 10 subdirectorios. En el mismo paso instalamos el paquete acl, que proporciona setfacl y getfacl para la sección 6.

La estructura es un recurso compartido del servidor, así que se crea una sola vez. La creó Guillermo (miembro A) desde la cuenta de administración. Alonso y Ariadna la comprueban con tree -d /srv/aurora6 desde su propia cuenta: cada uno ve solo los directorios a los que tiene acceso y el resto aparece como [error opening dir], lo que ya anticipa la política de la sección 6. Si alguien intentara volver a crearla, mkdir -p no daría error ni cambiaría nada.

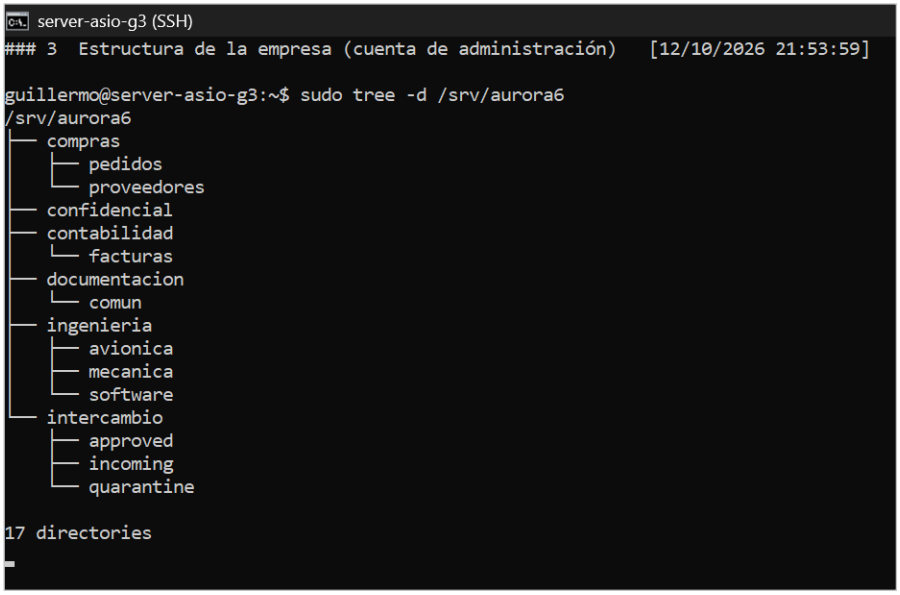


Figura 8. Estructura de /srv/aurora6 vista desde la cuenta de administración

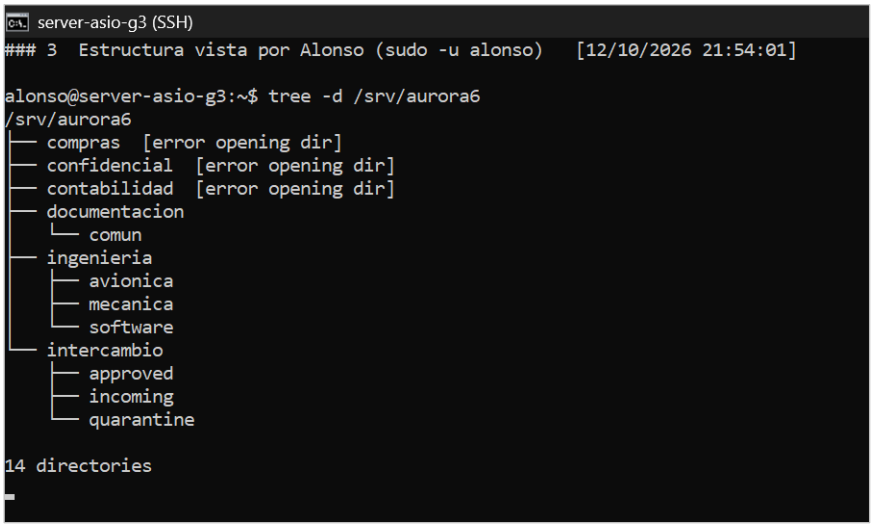


Figura 9. Estructura vista por Alonso

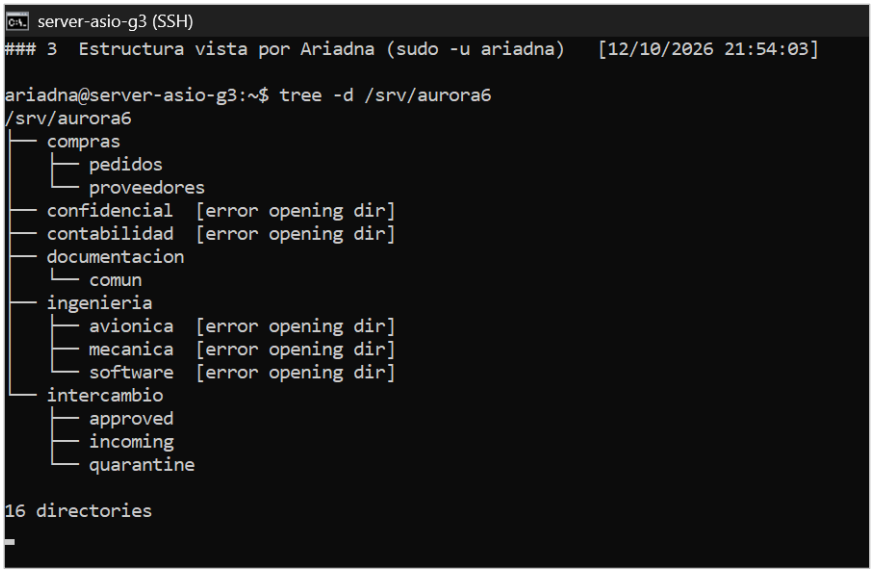


Figura 10. Estructura vista por Ariadna

4. Usuarios, grupos y roles

4.1. Creación de grupos y cuentas

Grupos y cuentas

```
sudo groupadd mecanica
sudo groupadd avionica
sudo groupadd software
sudo groupadd compras
sudo groupadd proyecto

sudo useradd -m -s /bin/bash usuario_a
sudo useradd -m -s /bin/bash alonso
sudo useradd -m -s /bin/bash ariadna

sudo passwd usuario_a
sudo passwd alonso
sudo passwd ariadna
```

`groupadd` crea cada grupo en `/etc/group` con un GID libre. `useradd` es la herramienta de bajo nivel: `-m` crea el directorio personal en `/home` copiando `/etc/skel` y `-s /bin/bash` fija el intérprete de órdenes (sin esa opción `useradd` asigna `/bin/sh`). En Ubuntu, `useradd` crea además un grupo privado con el mismo nombre que el usuario, que actúa como grupo primario. Hemos preferido `useradd` frente a `adduser` porque no es interactivo y por tanto es reproducible en un script; `adduser` es un envoltorio de Debian/Ubuntu que pregunta los datos y la contraseña. La contraseña se asigna con `passwd`, que la guarda cifrada (hash) en `/etc/shadow`; en `/etc/passwd` solo aparece una `x` en el campo de contraseña. Cada integrante escribe su propia contraseña y no se recoge en este informe.

4.2. Pertenencia a los grupos de departamento

Membresías

```
sudo usermod -aG mecanica,proyecto usuario_a
sudo usermod -aG avionica,software,proyecto alonso
sudo usermod -aG compras,proyecto ariadna
```

`usermod -G` establece la lista de grupos secundarios del usuario y `-a` (append) hace que se añadan a los que ya tenía. Sin `-a`, el usuario perdería cualquier grupo secundario que no aparezca en la lista, un error clásico que puede llegar a quitar el grupo `sudo` a una cuenta de administración. Hemos usado los departamentos como grupos secundarios y conservado el grupo privado como primario, porque B pertenece a dos departamentos y un usuario solo puede tener un grupo primario.

Las cuentas de trabajo no se añaden al grupo `sudo`. Es una decisión necesaria para la sección 6: si `usuario_a` pudiera usar `sudo`, podría acceder a `confidencial` y a cualquier otro directorio, y la política de permisos no tendría efecto.

4.3. Comprobación

Comprobación de cuentas y grupos

```
id usuario_a ; groups usuario_a
id alonso ; groups alonso
id ariadna ; groups ariadna
```

```
getent group mecanica avionica software compras proyecto
```

`id` muestra el UID, el grupo primario (`gid=`) y todos los grupos con su GID; `groups` muestra solo los nombres. `getent group` lee la base de datos de grupos y muestra, en el último campo, los miembros secundarios de cada uno. El resultado esperado es que `usuario_a` aparezca en `mecanica` y `proyecto`, `alonso` en `avionica`, `software` y `proyecto`, y `ariadna` en `compras` y `proyecto`. Los números UID y GID concretos dependen del sistema.

Un detalle importante: los grupos se cargan al iniciar sesión. Si un usuario ya tenía una sesión abierta cuando se le añadió a un grupo, en esa sesión `id` sin argumentos no mostrará el grupo nuevo y los accesos fallarán; hay que cerrar sesión y volver a entrar.

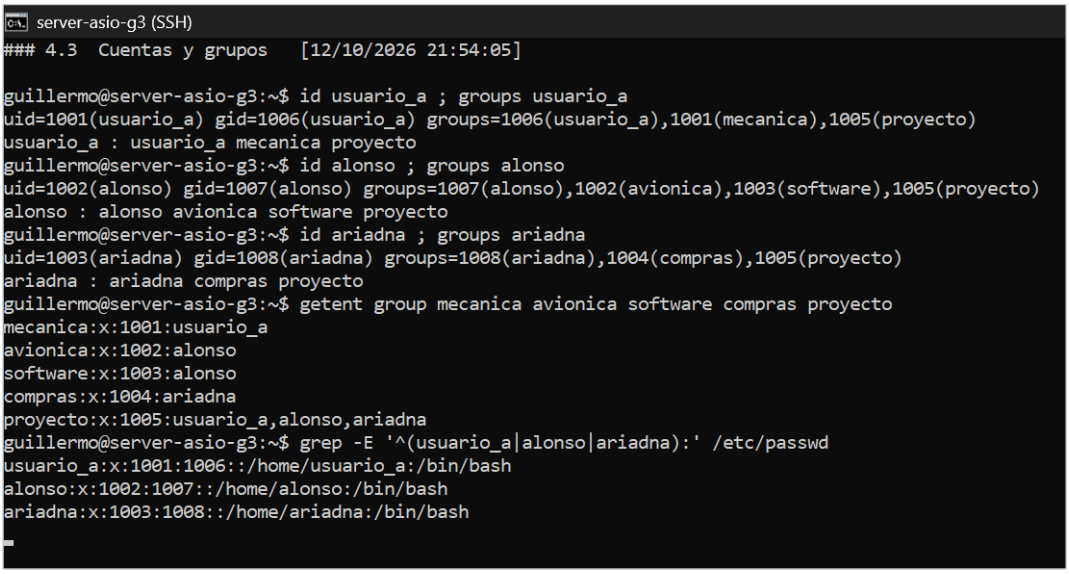


Figura 11. Cuentas, grupos y entradas de /etc/passwd

5. Permisos tradicionales (DAC) y configuración 751

Configuración temporal 751

```
sudo chown root:proyecto /srv/aurora6/documentacion
sudo chmod 751 /srv/aurora6/documentacion
ls -ld /srv/aurora6/documentacion
```

`chown root:proyecto` mantiene a root como propietario y asigna `proyecto` como grupo propietario. `chmod 751` fija los permisos en notación octal y `ls -ld` muestra los permisos del propio directorio (la opción `-d` evita que `ls` liste su contenido). El resultado esperado es una línea que empieza por `drwxr-x--x` con propietario `root` y grupo `proyecto`.

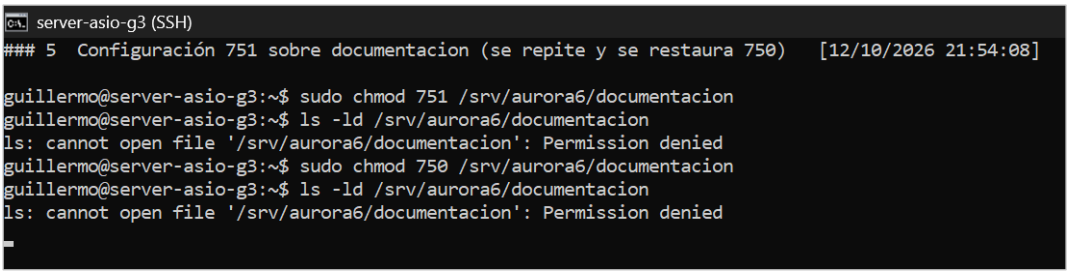


Figura 12. Configuración 751 sobre documentacion y vuelta a 750

5.1. Interpretación de 7, 5 y 1

Cada cifra octal es la suma de tres bits: lectura r = 4, escritura w = 2 y ejecución x = 1. La primera cifra se aplica al propietario, la segunda al grupo propietario y la tercera al resto de usuarios. La `d` inicial indica que se trata de un directorio.

Cifra	Clase	Bits	Significado en este directorio
7 = 4+2+1	Propietario (root)	rwx	Puede listar el contenido, crear, borrar y renombrar entradas, y entrar en él.
5 = 4+0+1	Grupo (proyecto)	r-x	Los miembros de proyecto pueden listar y entrar, pero no crear ni borrar archivos.
1 = 0+0+1	Otros	--x	El resto de usuarios puede atravesar el directorio para llegar a una ruta que conozca, pero no puede listarlo.

Tabla 4. Descomposición del permiso 751

En un directorio los permisos no significan lo mismo que en un archivo. La lectura permite leer la lista de nombres que contiene (`ls`), la escritura permite modificar esa lista, es decir, crear, borrar o renombrar entradas (y solo funciona si además se tiene x), y la ejecución es el permiso de paso o búsqueda: permite entrar con `cd` y acceder a los elementos del interior por su nombre. Por eso x es imprescindible para usar cualquier archivo de un directorio y de todos los directorios de la ruta hasta él. Con solo x, como los otros en 751, un usuario puede abrir `documentacion/comun/archivo.txt` si conoce el nombre y el archivo se lo permite, pero `ls documentacion` le devuelve Permission denied. Con solo r, en cambio, se verían los nombres pero no se podría entrar ni abrir nada.

El permiso 751 era una configuración temporal para practicar la interpretación. En la política definitiva (sección 6) `documentacion` pasa a 750, porque no hay ninguna razón para que usuarios ajenos al proyecto puedan atravesar el directorio.

6. Política de acceso por roles y ACLs

6.1. Traducción de la política

El primer paso ha sido convertir la política del enunciado en una matriz de accesos por directorio y por usuario. Para que un usuario llegue a un directorio necesita además el permiso x en todos los directorios de la ruta, así que también se han tenido en cuenta los niveles intermedios.

Directorio	usuario_a (A)	alonso (B)	ariadna (C)
ingenieria/mecanica	rwx	r-x	sin modificación (sin acceso)
ingenieria/avionica	r-x	rwx	sin modificación (sin acceso)
ingenieria/software	no indicado (sin acceso)	rwx	sin modificación (sin acceso)
compras (proveedores, pedidos)	sin acceso a pedidos	sin acceso a pedidos	rwx
contabilidad	sin acceso	sin acceso	solo paso (--x)
contabilidad/facturas	sin acceso	sin acceso	r-x
documentacion/comun	r-x	r-x	r-x
confidencial	sin acceso	sin acceso	sin acceso

Tabla 5. Matriz de accesos derivada del enunciado

Donde el enunciado no indica nada (por ejemplo, el acceso de A a `software` o de A y B a `proveedores`) hemos aplicado el principio de mínimo privilegio y no hemos concedido acceso. En el caso de C sobre ingeniería, el enunciado solo exige que no pueda modificar; denegarle también la lectura cumple ese requisito y es más restrictivo. A partir de la matriz se obtiene la configuración final:

Ruta	Propietario:grupo	Modo	ACL	Lo configura
/srv/aurora6	root:proyecto	750	no	Administración
ingenieria	root:proyecto	750	no	Administración
ingenieria/mecanica	root:mecanica	2770	u:alonso:r-x (+ por defecto)	A
ingenieria/avionica	root:avionica	2770	u:usuario_a:r-x (+ por defecto)	B
ingenieria/software	root:software	2770	no	B
compras, proveedores, pedidos	root:compras	2770	no	C
documentacion	root:proyecto	750	no	Administración
documentacion/comun	root:proyecto	2750	no	C
contabilidad	root:root	750	u:ariadna:--x	C
contabilidad/facturas	root:root	750	u:ariadna:r-x (+ por defecto)	C
confidencial	root:root	700	no	Administración

Ruta	Propietario:grupo	Modo	ACL	Lo configura
intercambio y subdirectorios	root:proyecto	750	no	Administración (se ampliará en P3-P5)

Tabla 6. Propietarios, permisos y ACLs finales

6.2. Criterios de diseño

Grupos de departamento como mecanismo principal. Cada directorio de departamento pertenece a root como propietario y al grupo del departamento, con permisos 770 para que el propietario y los miembros del grupo tengan control total y el resto ninguno. Hemos dejado a root como propietario en lugar del usuario del integrante para que los permisos del directorio no dependan de una persona: si mañana entra otro ingeniero mecánico basta con añadirlo al grupo `mecanica`.

Bit setgid (el 2 inicial). En un directorio, el bit setgid hace que los archivos y subdirectorios que se creen dentro hereden el grupo del directorio en vez del grupo primario de quien los crea. Sin él, un archivo que `usuario_a` crea en `mecanica` pertenecería al grupo privado `usuario_a` y los demás miembros del departamento no podrían trabajar con él. En `ls -ld` aparece como una `s` en la posición de ejecución del grupo (`drwxrws---`).

Niveles intermedios a 750 para el grupo proyecto. `/srv/aurora6`, `ingenieria` y `documentacion` dan r-x al grupo `proyecto` para que los tres integrantes puedan llegar a sus directorios, y nada a otros usuarios del sistema.

Confidencial a 700 de root. Ningún usuario de trabajo es propietario ni pertenece al grupo `root`, y los otros no tienen permisos, así que solo la cuenta de administración mediante `sudo` puede gestionarlo.

6.3. Cuándo es necesaria una ACL y qué aporta

El modelo propietario/grupo/otros solo permite definir permisos para un único usuario y un único grupo por archivo. En `mecanica` el grupo propietario ya está ocupado por `mecanica` (rwx) y necesitamos además que B tenga r-x, sin que los otros tengan nada (porque C no debe acceder). Con DAC solo habría dos salidas: dar r-x a los otros, lo que abriría el directorio también a C, o crear un grupo artificial que mezcle a A y a B, lo que daría a B los mismos derechos que a A. Ninguna cumple la política. Lo mismo ocurre en `avionica` con A y en `contabilidad/facturas` con C, ya que contabilidad debe seguir cerrada para A y B.

La ACL extendida añade entradas para usuarios o grupos concretos sin alterar el resto: `setfacl -m u:alonso:r-x mecanica` concede lectura y paso solo a B. La ventaja es la granularidad (permisos distintos para varios usuarios o grupos sobre el mismo recurso) sin crear grupos ficticios y sin abrir el recurso a todos los otros. Cuando un directorio tiene ACL, `ls -l` muestra un `+` al final de los permisos y los bits del grupo pasan a mostrar la máscara (`mask`), que es el límite máximo para los usuarios nombrados, los grupos nombrados y el grupo propietario. Si después se ejecuta un `chmod` que reduce los permisos del grupo, la máscara baja y la ACL deja de tener efecto aunque siga apareciendo; por eso hemos aplicado siempre `chmod` antes de `setfacl` y comprobado el resultado con `getfacl`, que indica con `#effective:` cuando la máscara recorta un permiso.

En los directorios con ACL hemos definido también una ACL por defecto (`setfacl -d`). La ACL de acceso solo afecta al directorio, mientras que la ACL por defecto se copia a los archivos y subdirectorios que se creen dentro. Sin ella, un archivo nuevo que A cree en `mecanica` no tendría la entrada de B y B no podría leerlo, aunque sí pudiera listar el directorio.

Un caso particular es `contabilidad`. C necesita llegar a `facturas`, pero el enunciado no le da acceso al resto de contabilidad. Con la entrada `u:ariadna:--x` C puede atravesar el directorio sin poder listarlo, que es exactamente el permiso de paso explicado en la sección 5.

6.4. Configuración realizada por cada integrante

Siguiendo el reparto del enunciado, cada integrante aplica su parte desde la cuenta de administración (las cuentas de trabajo no tienen `sudo`). Antes de ejecutar estos bloques la cuenta de administración aplica los permisos de los niveles comunes:

Niveles comunes y confidencial (administración)

```
sudo chown root:proyecto /srv/aurora6 /srv/aurora6/ingenieria /srv/aurora6/documentacion
/srv/aurora6/intercambio
sudo chmod 750 /srv/aurora6 /srv/aurora6/ingenieria /srv/aurora6/documentacion
/srv/aurora6/intercambio
sudo chown root:proyecto /srv/aurora6/intercambio/{incoming,approved,quarantine}
sudo chmod 750 /srv/aurora6/intercambio/{incoming,approved,quarantine}
sudo chown root:root /srv/aurora6/confidencial
sudo chmod 700 /srv/aurora6/confidencial
```

Miembro A: mecanica y ACL de lectura para B

```
sudo chown root:mecanica /srv/aurora6/ingenieria/mecanica
sudo chmod 2770 /srv/aurora6/ingenieria/mecanica
sudo setfacl -m u:alonso:r-x /srv/aurora6/ingenieria/mecanica
sudo setfacl -d -m u::rwx,g::rwx,o::---,u:alonso:r-x /srv/aurora6/ingenieria/mecanica
getfacl /srv/aurora6/ingenieria/mecanica
```

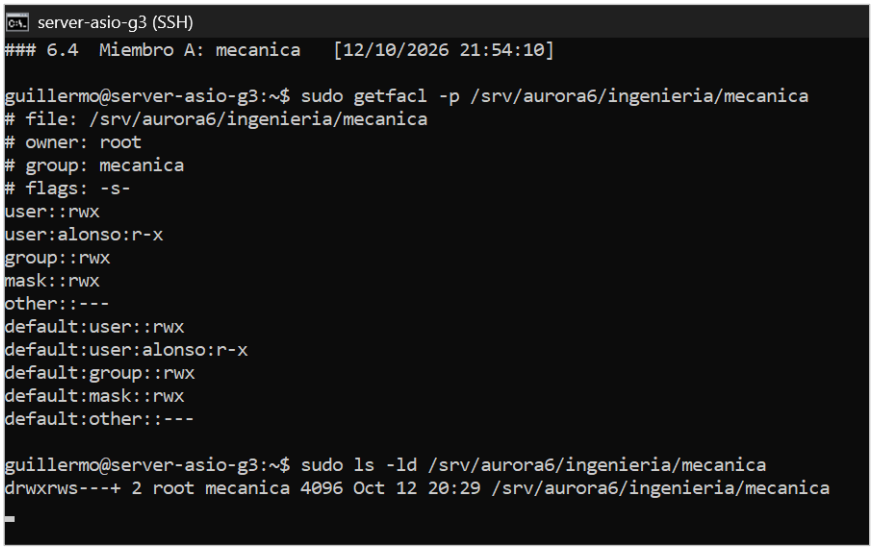


Figura 13. ACL de mecanica (miembro A)

Miembro B: avionica, software y ACL de lectura para A

```
sudo chown root:avionica /srv/aurora6/ingenieria/avionica
sudo chmod 2770 /srv/aurora6/ingenieria/avionica
sudo setfacl -m u:usuario_a:r-x /srv/aurora6/ingenieria/avionica
sudo setfacl -d -m u::rwx,g::rwx,o::---,u:usuario_a:r-x /srv/aurora6/ingenieria/avionica
sudo chown root:software /srv/aurora6/ingenieria/software
sudo chmod 2770 /srv/aurora6/ingenieria/software
getfacl /srv/aurora6/ingenieria/avionica
ls -ld /srv/aurora6/ingenieria/software
```

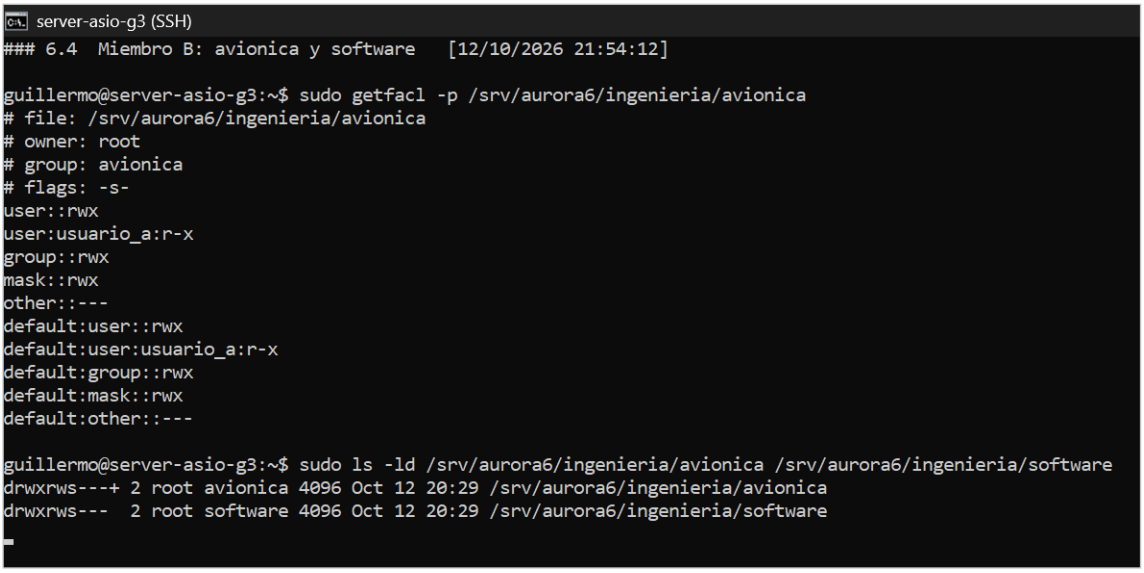


Figura 14. ACL de avionica y permisos de software (miembro B)

Miembro C: compras, documentacion/comun y ACL de contabilidad/facturas

```
sudo chown -R root:compras /srv/aurora6/compras
sudo chmod 2770 /srv/aurora6/compras /srv/aurora6/compras/proveedores
/srv/aurora6/compras/pedidos
sudo chown root:proyecto /srv/aurora6/documentacion/comun
sudo chmod 2750 /srv/aurora6/documentacion/comun
sudo chown root:root /srv/aurora6/contabilidad /srv/aurora6/contabilidad/facturas
sudo chmod 750 /srv/aurora6/contabilidad /srv/aurora6/contabilidad/facturas
sudo setfacl -m u:ariadna:--x /srv/aurora6/contabilidad
sudo setfacl -m u:ariadna:r-x /srv/aurora6/contabilidad/facturas
sudo setfacl -d -m u::rwx,g::r-x,o::---,u:ariadna:r-x /srv/aurora6/contabilidad/facturas
getfacl /srv/aurora6/contabilidad /srv/aurora6/contabilidad/facturas
```

El `chown -R` está limitado a `/srv/aurora6/compras`, que en ese momento solo contiene los dos subdirectorios vacíos, por lo que no afecta a ningún otro dato.

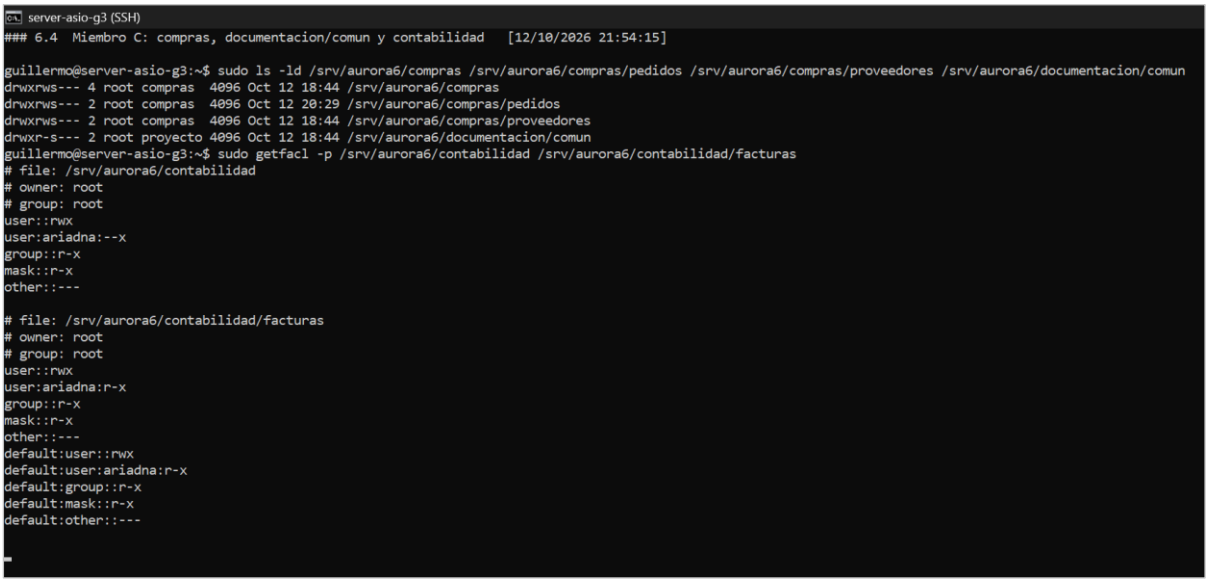


Figura 15. compras, documentacion/comun y ACL de contabilidad (miembro C)

6.5. Archivos de ejemplo para las pruebas

Para que las pruebas de lectura de la sección 7 no dependan del orden en que trabaje cada integrante, la cuenta de administración crea un archivo de ejemplo en cada recurso que se va a leer:

```
echo "Plan de ensayos estructurales AURORA-6" | sudo tee
/srv/aurora6/ingenieria/mecanica/plan_ensayos.txt
echo "Especificación de telemetría AURORA-6" | sudo tee
/srv/aurora6/ingenieria/avionica/telemetria.txt
echo "Normas comunes del proyecto AURORA-6" | sudo tee
/srv/aurora6/documentacion/comun/normas_proyecto.txt
echo "Factura 0001" | sudo tee /srv/aurora6/contabilidad/facturas/factura_0001.txt
sudo chmod 640 /srv/aurora6/documentacion/comun/normas_proyecto.txt
```

Se usa `sudo tee` porque una redirección `sudo echo ... > archivo` no funcionaría: la redirección la hace el shell del usuario sin privilegios antes de ejecutar `sudo`. En los directorios con ACL por defecto, el archivo nuevo hereda las entradas de la ACL (por ejemplo, `user:alonso` en mecanica) y la máscara se limita a rw- porque los archivos se crean sin bit de ejecución.

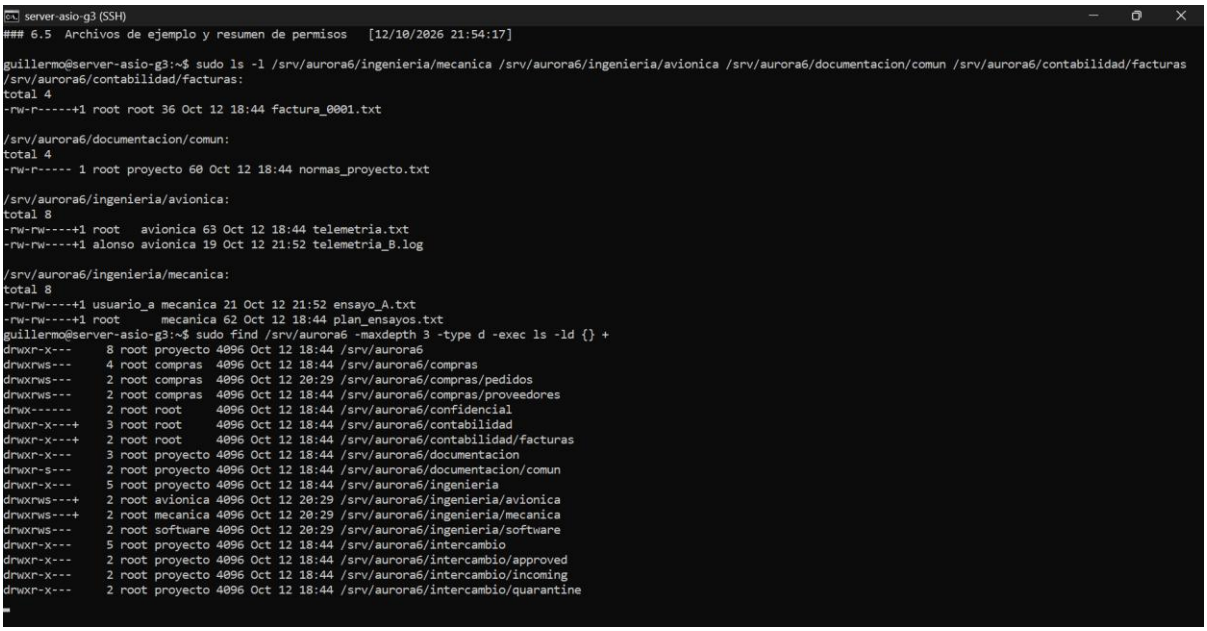


Figura 16. Archivos de ejemplo y resumen de propietarios y permisos de /srv/aurora6

7. Validación individual de la política de permisos

Cada prueba se ejecuta con la identidad del integrante correspondiente, y lo primero es comprobar con whoami e id que la sesión corresponde a su usuario y que ya incluye sus grupos. Las salidas de esta sección se obtuvieron en el servidor el 12/10/2026 ejecutando cada orden con la cuenta de cada integrante mediante sudo -u <cuenta> desde la cuenta de administración. El núcleo aplica las mismas comprobaciones de permisos que en una sesión SSH de ese usuario, porque el proceso se ejecuta con su UID y sus grupos, como confirma la salida de id. Cuando la regla de reenvío y el firewall estén abiertos, cada integrante puede repetirlas desde su PC con ssh -p 2222 <cuenta>@[IP_PC_servidor]. El sistema está en inglés, así que los errores aparecen como «Permission denied».

En las tablas, el resultado esperado se dedujo de la configuración de la sección 6 y la columna de resultado obtenido resume la salida real del servidor, que se muestra completa en las capturas que siguen a cada tabla. Las capturas se tomaron el 12/10/2026 en una sesión SSH de la cuenta de administración; la cabecera de cada pantalla indica la sección, la cuenta con la que se ejecutan las órdenes y la hora del servidor.

7.1. Pruebas del miembro A: Guillermo (usuario_a)

Comprobación de identidad al iniciar sesión

```
whoami
id
```

Prueba	Comando	Resultado esperado	Resultado obtenido
Escritura permitida en su departamento	`echo "Ensayo de vibración" > /srv/aurora6/ingenieria/mecanica/ensayo_A.txt` `ls -l /srv/aurora6/ingenieria/mecanica/ensayo_A.txt`	Se crea el archivo, propietario usuario_a, grupo mecanica (setgid), con + por la ACL por defecto	Se crea el archivo: -rw-rw----+ 1 usuario_a mecanica 21 ... ensayo_A.txt (código 0). Coincide.
Lectura permitida en departamento ajeno	`ls -l /srv/aurora6/ingenieria/avionica` `cat /srv/aurora6/ingenieria/avionica/telemetria.txt`	Lista el contenido y muestra el texto del archivo	Lista telemetria.txt y muestra «Especificación de telemetría AURORA-6 (documento de ejemplo)». Coincide.
Escritura que debe rechazarse	`touch /srv/aurora6/ingenieria/avionica/intento_A.txt`	touch: cannot touch ...: Permission denied (la ACL solo da r-x)	touch: cannot touch '/srv/aurora6/ingenieria/avionica/intento_A.txt': Permission denied (código 1). Coincide.
Acceso a directorio prohibido	`ls /srv/aurora6/contabilidad` `cd /srv/aurora6/confidencial`	Permission denied en ambos casos	ls: cannot open directory '/srv/aurora6/contabilidad' : Permission denied; cd: /srv/aurora6/confidencial: Permission denied. Coincide.
Acceso prohibido a pedidos	`ls /srv/aurora6/compras/pedidos`	Permission denied (no tiene x en compras)	ls: cannot open file '/srv/aurora6/compras/pedidos': Permission denied (código 2). Coincide.

Prueba	Comando	Resultado esperado	Resultado obtenido
Lectura de documentación común	`cat /srv/aurora6/documentacion/comun/normas_proyecto.txt`	Muestra el texto	Muestra «Normas comunes del proyecto AURORA-6 (documento de ejemplo)». Coincide.
ACL más relevante para su rol	`getfacl /srv/aurora6/ingenieria/mecanica`	Aparece `user:alonso:r-x` y `group::rwx`	Aparecen user:alonso:r-x, group::rwx, mask::rwx, other::--- y las líneas default:. Coincide.

Tabla 7. Pruebas de usuario_a

```
server-asio-g3 (SSH)
### 7.1  Pruebas de usuario_a (Guillermo) 1/2 - sudo -u usuario_a   [12/10/2026 21:54:19]

usuario_a@server-asio-g3:~$ whoami
usuario_a
usuario_a@server-asio-g3:~$ id
uid=1001(usuario_a) gid=1006(usuario_a) groups=1006(usuario_a),1001(mecanica),1005(proyecto)
usuario_a@server-asio-g3:~$ echo 'Ensayo de vibración' > /srv/aurora6/ingenieria/mecanica/ensayo_A.txt
usuario_a@server-asio-g3:~$ ls -l /srv/aurora6/ingenieria/mecanica/ensayo_A.txt
-rw-rw----+ 1 usuario_a mecanica 21 Oct 12 21:54 /srv/aurora6/ingenieria/mecanica/ensayo_A.txt
usuario_a@server-asio-g3:~$ ls -l /srv/aurora6/ingenieria/avionica
total 8
-rw-rw----+1 root   avionica 63 Oct 12 18:44 telemetria.txt
-rw-rw----+1 alonso avionica 19 Oct 12 21:52 telemetria_B.log
usuario_a@server-asio-g3:~$ cat /srv/aurora6/ingenieria/avionica/telemetria.txt
Especificación de telemetría AURORA-6 (documento de ejemplo)
usuario_a@server-asio-g3:~$ touch /srv/aurora6/ingenieria/avionica/intento_A.txt
touch: cannot touch '/srv/aurora6/ingenieria/avionica/intento_A.txt': Permission denied
```

Figura 17. Pruebas de usuario_a (1/2)

```
server-asio-g3 (SSH)
### 7.1  Pruebas de usuario_a (Guillermo) 2/2 - sudo -u usuario_a   [12/10/2026 21:54:22]

usuario_a@server-asio-g3:~$ ls /srv/aurora6/contabilidad
ls: cannot open directory '/srv/aurora6/contabilidad': Permission denied
usuario_a@server-asio-g3:~$ cd /srv/aurora6/confidencial
bash: line 1: cd: /srv/aurora6/confidencial: Permission denied
usuario_a@server-asio-g3:~$ ls /srv/aurora6/compras/pedidos
ls: cannot open file '/srv/aurora6/compras/pedidos': Permission denied
usuario_a@server-asio-g3:~$ cat /srv/aurora6/documentacion/comun/normas_proyecto.txt
Normas comunes del proyecto AURORA-6 (documento de ejemplo)
usuario_a@server-asio-g3:~$ getfacl -p /srv/aurora6/ingenieria/mecanica
# file: /srv/aurora6/ingenieria/mecanica
# owner: root
# group: mecanica
# flags: -s-
user::rwx
user:alonso:r-x
group::rwx
mask::rwx
other::---
default:user::rwx
default:user:alonso:r-x
default:group::rwx
default:mask::rwx
default:other::---
```

Figura 18. Pruebas de usuario_a (2/2)

7.2. Pruebas del miembro B: Alonso (alonso)

Comprobación de identidad al iniciar sesión

```
whoami
id
```

Prueba	Comando	Resultado esperado	Resultado obtenido

Escritura permitida en su departamento	`echo "Log de telemetría" > /srv/aurora6/ingenieria/avionica/telemetria_B.log`
	`echo "print('control')" > /srv/aurora6/ingenieria/software/control_B.py`
	`ls -l /srv/aurora6/ingenieria/avionica /srv/aurora6/ingenieria/software`
		Se crean ambos archivos, con grupo avionica y software respectivamente	Se crean telemetria_B.log (alonso avionica, con +) y control_B.py (alonso software), código 0. Coincide.

Lectura permitida en departamento ajeno	`ls -l /srv/aurora6/ingenieria/mecanica` `cat /srv/aurora6/ingenieria/mecanica/plan_ensayos.txt`	Lista el contenido y muestra el texto	Lista plan_ensayos.txt y ensayo_A.txt y muestra el texto de plan_ensayos.txt; también lee ensayo_A.txt, creado por A, gracias a la ACL por defecto. Coincide.
Escritura que debe rechazarse	`touch /srv/aurora6/ingenieria/mecanica/intento_B.txt`	touch: cannot touch ...: Permission denied	touch: cannot touch '/srv/aurora6/ingenieria/mecanica/intento_B.txt': Permission denied (código 1). Coincide.
Acceso a directorio prohibido	`ls /srv/aurora6/compras/pedidos` `ls /srv/aurora6/confidencial`	Permission denied en ambos casos	ls: cannot open file '/srv/aurora6/compras/pedidos': Permission denied; ls: cannot open directory '/srv/aurora6/confidencial': Permission denied. Coincide.
Lectura de documentación común	`cat /srv/aurora6/documentacion/comun/normas_proyecto.txt`	Muestra el texto	Muestra «Normas comunes del proyecto AURORA-6 (documento de ejemplo)». Coincide.
ACL más relevante para su rol	`getfacl /srv/aurora6/ingenieria/avionica`	Aparece `user:usuario_a:r-x` y `group::rwx`	Aparecen user:usuario_a:r-x, group::rwx, mask::rwx y las líneas default:. Coincide.

Tabla 8. Pruebas de alonso

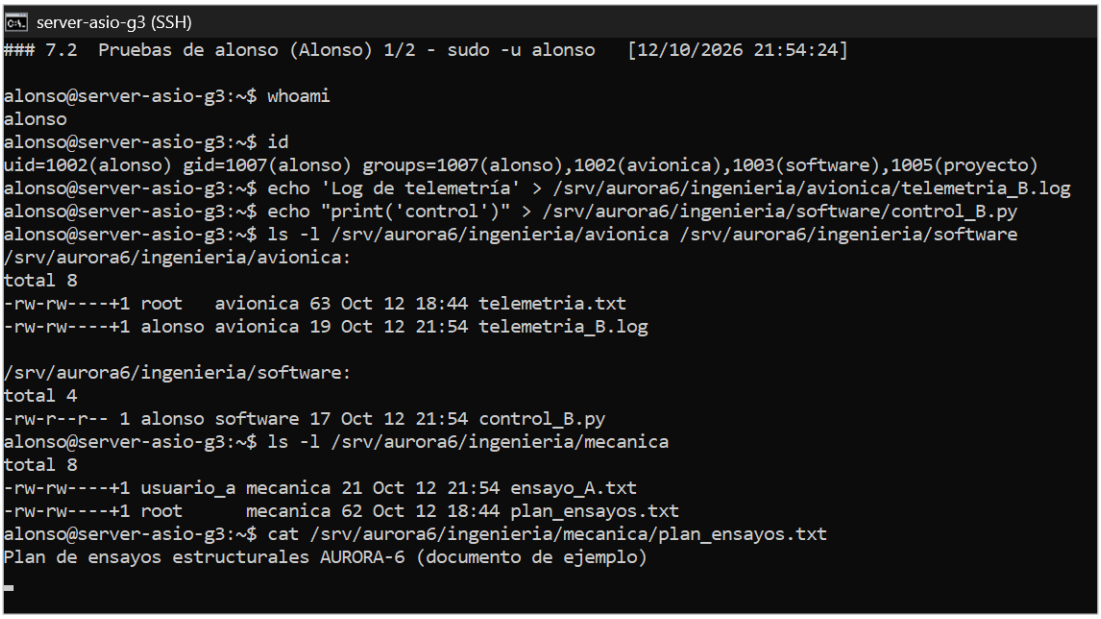


Figura 19. Pruebas de alonso (1/2)

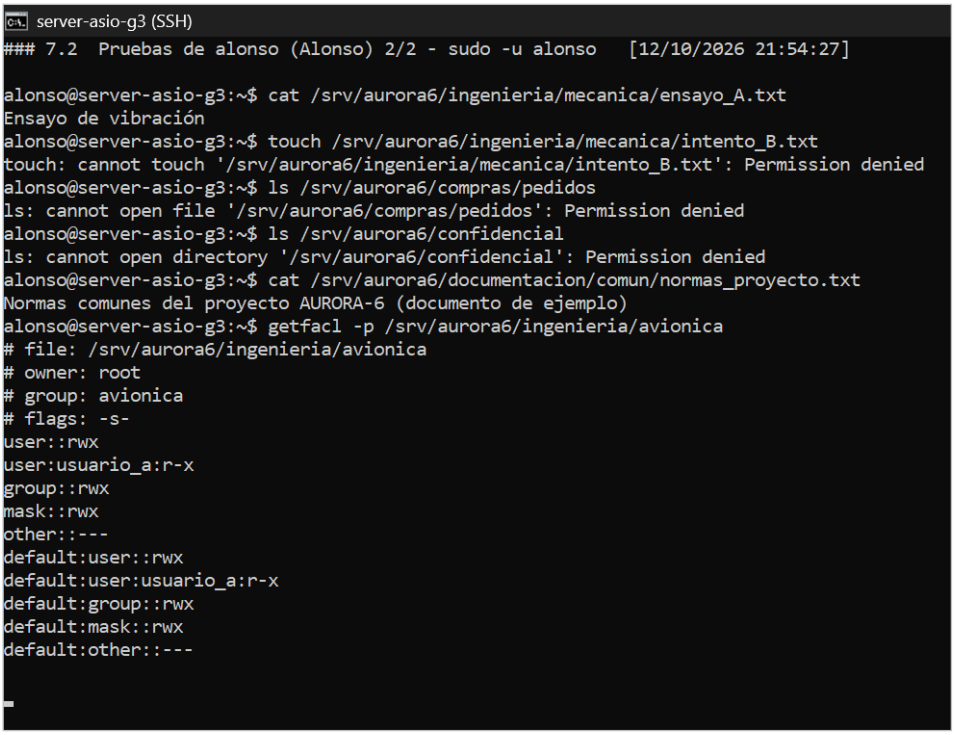


Figura 20. Pruebas de alonso (2/2)

7.3. Pruebas del miembro C: Ariadna (ariadna)

Comprobación de identidad al iniciar sesión

```
whoami
id
```

Prueba	Comando	Resultado esperado	Resultado obtenido
Escritura permitida en su departamento	`echo "Pedido 001: tornillería" > /srv/aurora6/compras/pedidos/pedido_001.txt` `ls -l /srv/aurora6/compras/pedidos`	Se crea el archivo, propietario ariadna, grupo compras	Se crea el archivo: -rw-r--r-- 1 ariadna compras 25 ... pedido_001.txt (código 0). Coincide.
Lectura permitida en departamento ajeno	`cat /srv/aurora6/contabilidad/facturas/factura_0001.txt` `cat /srv/aurora6/documentacion/comun/normas_proyecto.txt`	Muestra el texto de ambos archivos	Muestra «Factura 0001 (documento de ejemplo)» y «Normas comunes del proyecto AURORA-6 (documento de ejemplo)». Coincide.
Paso sin listado en contabilidad	`ls /srv/aurora6/contabilidad`	Permission denied: solo tiene --x, puede atravesar pero no listar	ls: cannot open directory '/srv/aurora6/contabilidad' : Permission denied (código 2), aunque sí llega a facturas. Coincide.
Escritura que debe rechazarse	`touch /srv/aurora6/ingenieria/mecanica/intento_C.txt` `touch /srv/aurora6/contabilidad/facturas/intento_C.txt`	Permission denied en ambos casos	touch: setting times of '.../mecanica/intento_C.txt' : Permission denied; touch: cannot touch '.../facturas/intento_C.txt': Permission denied. Coincide.
Acceso a directorio prohibido	`cd /srv/aurora6/confidencial`	Permission denied	bash: cd: /srv/aurora6/confidencial: Permission denied (código 1). Coincide.
ACL más relevante para su rol	`getfacl /srv/aurora6/contabilidad/facturas`	Aparece `user:ariadna:r-x` y las entradas `default:`	Aparecen user:ariadna:r-x, mask::r-x y las entradas default:user:ariadna:r-x. Coincide.

Tabla 9. Pruebas de ariadna



Figura 21. Pruebas de ariadna (1/2)

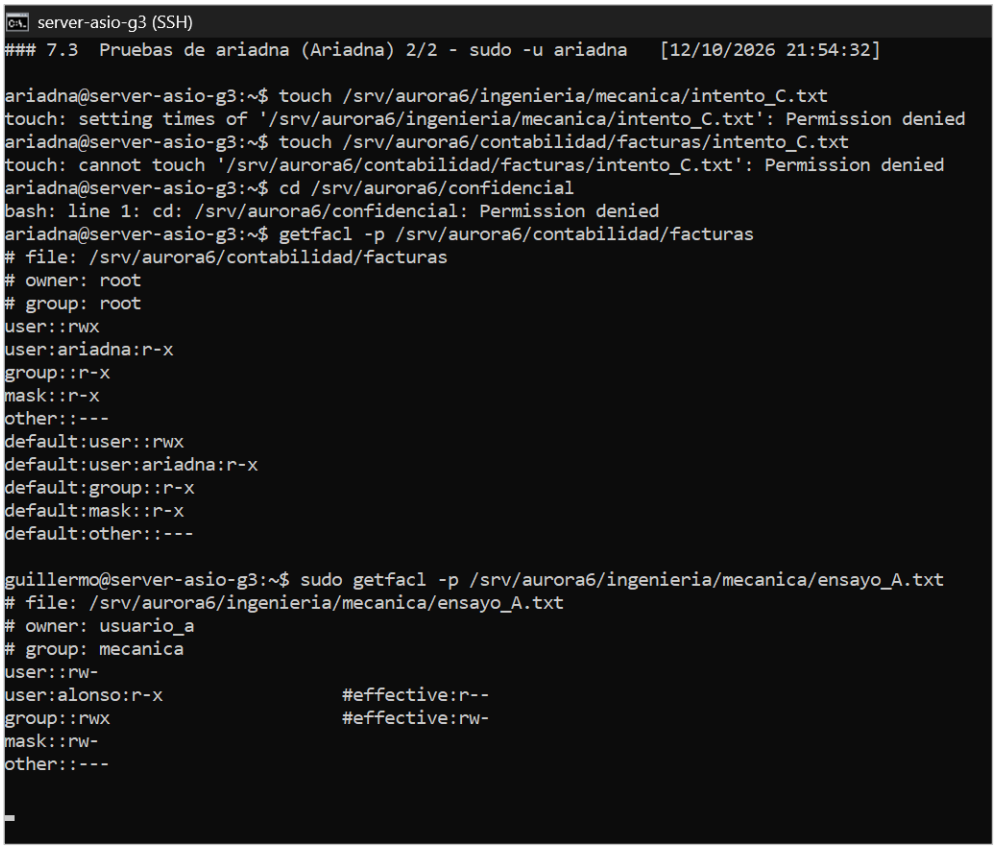


Figura 22. Pruebas de ariadna (2/2) y herencia de la ACL por defecto en ensayo_A.txt

7.4. Interpretación

Las pruebas demuestran el comportamiento real del núcleo y no solo la configuración mostrada por `ls -l`. Las escrituras en el departamento propio funcionan porque el usuario es miembro del grupo propietario con rwx. Las lecturas cruzadas entre mecánica y aviónica funcionan gracias a la entrada de ACL r-x, y la escritura cruzada falla porque esa entrada no incluye w. Los accesos a `confidencial`, `contabilidad` y `compras/pedidos` fallan porque el usuario cae en la clase otros, que no tiene ningún permiso, o porque le falta el permiso x en un directorio de la ruta. En el caso de C, la combinación de `ls contabilidad` denegado y `cat contabilidad/facturas/factura_0001.txt` permitido muestra la diferencia entre los permisos r y x de un directorio.

Todos los resultados obtenidos coinciden con los esperados. La salida de getfacl de ensayo_A.txt confirma además la herencia de la ACL por defecto: el archivo que creó A en mecanica tiene la entrada user:alonso:r-x, que la máscara limita a r--, y por eso Alonso puede leerlo. En la prueba de Ariadna sobre mecanica, touch informa «setting times of ...» en lugar de «cannot touch», pero el resultado es el mismo: Permission denied, código de salida 1 y ningún archivo creado. Por último, los archivos que Alonso crea en software y Ariadna en pedidos quedan con permisos 644 (-rw-r--r--), que es lo que produce la umask 022 por defecto en directorios sin ACL por defecto. No supone un riesgo, porque el modo 2770 del directorio impide el acceso a los otros, pero sí impide que los compañeros del mismo grupo modifiquen esos archivos; en las prácticas siguientes conviene usar una umask 007 o una ACL por defecto también en software y compras.

8. Incidencias y soluciones

La tabla recoge las situaciones que pueden aparecer con más probabilidad al aplicar esta configuración y, en las cuatro últimas filas, las incidencias reales que tuvimos durante la práctica, con su causa y su solución.

Síntoma	Causa	Comprobación y solución
Un usuario recibe Permission denied en su propio departamento	La sesión se abrió antes de añadirlo al grupo	`id` sin argumentos no muestra el grupo. Cerrar sesión y volver a entrar.
La ACL aparece en `getfacl` pero no tiene efecto	Un `chmod` posterior redujo la máscara	`getfacl` muestra `#effective:---`. Restaurar con `setfacl -m m::rwx ruta` o repetir el `chmod` y el `setfacl` en ese orden.
`setfacl: command not found`	Falta el paquete acl	`sudo apt install acl`
B lista mecanica pero no puede leer un archivo nuevo de A	Falta la ACL por defecto en el directorio	`getfacl` del directorio no muestra líneas `default:`. Aplicar `setfacl -d -m ...` y corregir los archivos existentes con `setfacl -m u:alonso:r--`.
Un usuario quita el grupo `sudo` a la cuenta de administración	`usermod -G` sin `-a`	Recuperar desde otra cuenta con sudo o desde el modo de recuperación con `usermod -aG sudo <admin>`.
El cliente no conecta por SSH (timeout)	Regla de reenvío ligada a 127.0.0.1, firewall de Windows o aislamiento de la red	`Test-NetConnection [IP] -Port 2222` desde el cliente; revisar la regla de VirtualBox y la regla del firewall; usar el punto de acceso de un móvil si la red aísla los equipos.
`Connection refused` en el puerto 2222	La VM está apagada o SSH no escucha	`sudo ss -tlnp | grep :22` y `systemctl status ssh.socket ssh` en la VM.
La VM se queda bloqueada al arrancar (en el initramfs) y el puerto 2222 rechaza la conexión	Integridad de memoria y el hipervisor de Windows (usado por WSL2) estaban activos, así que VirtualBox no podía usar VT-x y funcionaba sobre Hyper-V. VBox.log mostraba «Attempting fall back to NEM: VT-x is not available».	Desactivar Seguridad de Windows > Aislamiento del núcleo > Integridad de memoria y ejecutar bcdedit /set hypervisorlaunchtype off como administrador, y reiniciar. Después VBox.log muestra «VT-x w/ nested paging» y la VM arranca. Se revierte con bcdedit /set hypervisorlaunchtype auto.
La VM se cierra sola al reanudarla desde un estado guardado	El estado se había guardado con la VM bloqueada en el arranque; el proceso de VirtualBox terminó con el código 0xc0000005	Descartar el estado guardado y arrancar la VM en frío.
ssh -p 2222 guillermo@localhost falla aunque la VM está encendida	En Windows, localhost se resuelve primero a ::1 (IPv6) y la regla de reenvío de VirtualBox solo escucha en 127.0.0.1	Conectar con ssh -p 2222 guillermo@127.0.0.1 o dejar vacío el campo IP anfitrión de la regla.

Síntoma	Causa	Comprobación y solución
Las cuentas de B y C se crearon como usuario_b y usuario_c y el grupo acordó usar alonso y ariadna	Cambio de nombres posterior a la creación de la estructura y de las ACL	sudo usermod -l alonso -d /home/alonso -m usuario_b y sudo groupmod -n alonso usuario_b (igual con ariadna). Las ACL guardan el UID, por lo que getfacl muestra ya el nombre nuevo sin repetir setfacl.

Tabla 10. Incidencias previstas y reales

9. Conclusiones

La práctica nos ha obligado a pasar de una política escrita en lenguaje de empresa a una configuración concreta del sistema de archivos. El paso más importante fue construir primero la matriz de accesos por usuario y directorio, incluidos los directorios intermedios, porque permite ver qué requisitos se cubren con grupos y cuáles no.

Los grupos de departamento resuelven la mayoría de los casos y son la opción más mantenible: dar de alta a un nuevo empleado consiste en añadirlo a un grupo. Las ACL han sido necesarias solo en tres puntos, cuando un usuario externo al grupo necesitaba un permiso distinto del de los demás sin abrir el recurso a todos (B en mecánica, A en aviónica y C en facturas). Hemos comprobado que una ACL de acceso sin ACL por defecto se queda corta para los archivos nuevos, y que la máscara puede anular una ACL sin que sea evidente a simple vista, por lo que `getfacl` es imprescindible para verificar.

También ha quedado clara la diferencia entre los permisos de un directorio y los de un archivo, en especial el papel del permiso x como permiso de paso, que hemos aprovechado para que C llegue a las facturas sin poder ver el resto de la contabilidad. Por último, mantener las cuentas de trabajo sin `sudo` es lo que da sentido a toda la política: el directorio confidencial solo es realmente confidencial si ningún usuario de trabajo puede elevar privilegios.

10. Bibliografía

- Sampedro Lirio, D. Administración de Servicios de Sistemas Operativos (88020). Enunciados de prácticas P1 y P2 y apuntes del Tema 1. Universidad Camilo José Cela.
- Páginas de manual de Ubuntu: chmod(1), chown(1), useradd(8), usermod(8), groupadd(8), setfacl(1), getfacl(1) y acl(5). Disponibles con `man` en el propio servidor y en https://manpages.ubuntu.com/.
- Canonical. Ubuntu Server documentation. https://documentation.ubuntu.com/server/
- Linux Foundation. Filesystem Hierarchy Standard 3.0. https://refspecs.linuxfoundation.org/fhs.shtml

Anexo A. Script de configuración del servidor

El script `p2_servidor.sh` reúne todos los comandos de las secciones 3 a 6 en bloques independientes, para que cada integrante pueda ejecutar su parte (`sudo bash p2_servidor.sh politica_A`) o para reconstruir el servidor completo (`sudo bash p2_servidor.sh todo`). Es repetible: se puede ejecutar varias veces sin duplicar usuarios ni grupos. `set -euo pipefail` detiene el script ante el primer error, ante variables sin definir y ante fallos dentro de una tubería. Las contraseñas no se incluyen y se asignan de forma interactiva con `passwd`.

```
#!/bin/bash
# ASSIO (88020) - Práctica 2: estructura, usuarios, grupos, DAC y ACLs
# Proyecto AURORA-6. Se ejecuta en el servidor Ubuntu con la cuenta de administración:
#   sudo bash p2_servidor.sh <bloque>
# Bloques: estructura | usuarios | dac751 | base | politica_A | politica_B | politica_C |
semillas | verificar | todo

set -euo pipefail

# Nombres de las cuentas de trabajo (cámbialos por los acordados en el grupo)
USER_A="usuario_a"   # Miembro A: Ingeniería mecánica
USER_B="alonso"   # Miembro B: Aviónica y software
USER_C="ariadna"   # Miembro C: Compras y configuración

BASE="/srv/aurora6"

if [ "$(id -u)" -ne 0 ]; then
    echo "Ejecuta el script con sudo: sudo bash $0 <bloque>"
    exit 1
fi

for cmd in setfacl getfacl; do
    if ! command -v "$cmd" >/dev/null 2>&1; then
        echo "Falta $cmd. Instálalo con: sudo apt install acl"
        exit 1
    fi
done

paso() { echo; echo "==> $*"; }

estructura() {
    paso "Creando la estructura de $BASE"
    mkdir -p "$BASE"/ingenieria/{mecanica,avionica,software} \
             "$BASE"/compras/{proveedores,pedidos} \
             "$BASE"/contabilidad/facturas \
             "$BASE"/documentacion/comun \
             "$BASE"/confidencial \
             "$BASE"/intercambio/{incoming,approved,quarantine}
    tree -d "$BASE" 2>/dev/null || find "$BASE" -type d | sort
}

usuarios() {
    paso "Creando grupos de departamento y grupo común"
    for g in mecanica avionica software compras proyecto; do
        getent group "$g" >/dev/null || groupadd "$g"
    done

    paso "Creando cuentas de trabajo (sin privilegios sudo)"
    for u in "$USER_A" "$USER_B" "$USER_C"; do
        id "$u" >/dev/null 2>&1 || useradd -m -s /bin/bash "$u"
    done
```

```
    usermod -aG mecanica,proyecto          "$USER_A"
    usermod -aG avionica,software,proyecto "$USER_B"
    usermod -aG compras,proyecto           "$USER_C"

    for u in "$USER_A" "$USER_B" "$USER_C"; do
        id "$u"
        groups "$u"
    done
    echo
    echo "Asigna ahora la contraseña de cada cuenta de forma interactiva:"
    echo "  sudo passwd $USER_A ; sudo passwd $USER_B ; sudo passwd $USER_C"
}

dac751() {
    paso "Ejercicio 751 sobre documentacion"
    chown root:proyecto "$BASE/documentacion"
    chmod 751 "$BASE/documentacion"
    ls -ld "$BASE/documentacion"
}

base() {
    paso "Permisos base de los niveles superiores"
    chown root:proyecto "$BASE" "$BASE/ingenieria" "$BASE/documentacion"
"$BASE/intercambio"
    chmod 750           "$BASE" "$BASE/ingenieria" "$BASE/documentacion"
"$BASE/intercambio"
    chown root:proyecto "$BASE"/intercambio/{incoming,approved,quarantine}
    chmod 750           "$BASE"/intercambio/{incoming,approved,quarantine}

    paso "confidencial: solo root (gestión mediante sudo)"
    chown root:root "$BASE/confidencial"
    chmod 700 "$BASE/confidencial"
    ls -ld "$BASE" "$BASE"/* "$BASE/confidencial"
}

politica_A() {
    paso "[Miembro A] mecanica: grupo mecanica rwx, ACL de lectura para $USER_B"
    chown root:mecanica "$BASE/ingenieria/mecanica"
    chmod 2770 "$BASE/ingenieria/mecanica"
    setfacl -m u:"$USER_B":r-x "$BASE/ingenieria/mecanica"
    setfacl -d -m u::rwx,g::rwx,o::---,u:"$USER_B":r-x "$BASE/ingenieria/mecanica"
    getfacl "$BASE/ingenieria/mecanica"
}

politica_B() {
    paso "[Miembro B] avionica: grupo avionica rwx, ACL de lectura para $USER_A"
    chown root:avionica "$BASE/ingenieria/avionica"
    chmod 2770 "$BASE/ingenieria/avionica"
    setfacl -m u:"$USER_A":r-x "$BASE/ingenieria/avionica"
    setfacl -d -m u::rwx,g::rwx,o::---,u:"$USER_A":r-x "$BASE/ingenieria/avionica"
    getfacl "$BASE/ingenieria/avionica"

    paso "[Miembro B] software: grupo software rwx"
    chown root:software "$BASE/ingenieria/software"
    chmod 2770 "$BASE/ingenieria/software"
    ls -ld "$BASE/ingenieria/software"
}

politica_C() {
    paso "[Miembro C] compras: grupo compras rwx (incluye proveedores y pedidos)"
    chown -R root:compras "$BASE/compras"
    chmod 2770 "$BASE/compras" "$BASE/compras/proveedores" "$BASE/compras/pedidos"
    ls -ld "$BASE/compras" "$BASE"/compras/*
```

```
    paso "[Miembro C] documentacion/comun: lectura para el grupo proyecto"
    chown root:proyecto "$BASE/documentacion/comun"
    chmod 2750 "$BASE/documentacion/comun"
    ls -ld "$BASE/documentacion/comun"

    paso "[Miembro C] contabilidad: cerrada; ACL de paso para $USER_C y de lectura en
facturas"
    chown root:root "$BASE/contabilidad" "$BASE/contabilidad/facturas"
    chmod 750 "$BASE/contabilidad" "$BASE/contabilidad/facturas"
    setfacl -m u:"$USER_C":--x "$BASE/contabilidad"
    setfacl -m u:"$USER_C":r-x "$BASE/contabilidad/facturas"
    setfacl -d -m u::rwx,g::r-x,o::---,u:"$USER_C":r-x "$BASE/contabilidad/facturas"
    getfacl "$BASE/contabilidad" "$BASE/contabilidad/facturas"
}

semillas() {
    paso "Creando archivos de ejemplo para las pruebas de lectura"
    echo "Plan de ensayos estructurales AURORA-6 (documento de ejemplo)" >
"$BASE/ingenieria/mecanica/plan_ensayos.txt"
    echo "Especificación de telemetría AURORA-6 (documento de ejemplo)"  >
"$BASE/ingenieria/avionica/telemetria.txt"
    echo "Normas comunes del proyecto AURORA-6 (documento de ejemplo)"  >
"$BASE/documentacion/comun/normas_proyecto.txt"
    echo "Factura 0001 (documento de ejemplo)"                          >
"$BASE/contabilidad/facturas/factura_0001.txt"
    echo "Documento reservado (solo administración)"                    >
"$BASE/confidencial/reservado.txt"
    chmod 640 "$BASE/documentacion/comun/normas_proyecto.txt"
    chmod 600 "$BASE/confidencial/reservado.txt"
    ls -l "$BASE/ingenieria/mecanica" "$BASE/ingenieria/avionica" \
          "$BASE/documentacion/comun" "$BASE/contabilidad/facturas"
}

verificar() {
    paso "Resumen de propietarios, permisos y ACLs"
    find "$BASE" -maxdepth 3 -type d -exec ls -ld {} +
    echo
    getfacl -R -p "$BASE" 2>/dev/null | grep -v '^$' | grep -E '^#
file|user:|group:|mask|other' || true
}

case "${1:-}" in
    estructura) estructura ;;
    usuarios)   usuarios ;;
    dac751)     dac751 ;;
    base)       base ;;
    politica_A) politica_A ;;
    politica_B) politica_B ;;
    politica_C) politica_C ;;
    semillas)   semillas ;;
    verificar)  verificar ;;
    todo)       estructura; usuarios; dac751; base; politica_A; politica_B; politica_C;
semillas; verificar ;;
    *) echo "Uso: sudo bash $0
{estructura|usuarios|dac751|base|politica_A|politica_B|politica_C|semillas|verificar|todo}"
; exit 1 ;;
esac
```

Anexo B. Preguntas previsibles en la defensa

Pregunta	Respuesta breve
¿Qué diferencia hay entre r y x en un directorio?	r permite listar los nombres; x permite entrar y acceder a los elementos por su nombre. C en contabilidad tiene solo x: llega a facturas pero no puede listar contabilidad.
¿Por qué 2770 y no 770?	El 2 activa setgid: los archivos nuevos heredan el grupo del directorio, así todo lo que se crea en mecanica pertenece al grupo mecanica.
¿Por qué no basta con permisos DAC en mecanica?	El grupo ya es mecanica (rwx) y B necesita r-x sin abrir el directorio a otros (C). DAC solo admite un grupo; la ACL añade un usuario concreto.
¿Qué es la máscara de una ACL?	El permiso máximo efectivo para usuarios y grupos nombrados y el grupo propietario. `ls -l` muestra la máscara en la posición del grupo.
¿Qué pasa si quitas `-a` en `usermod -aG`?	El usuario pierde los grupos secundarios que no estén en la lista.
¿Dónde están las contraseñas?	En `/etc/shadow`, como hash, legible solo por root. `/etc/passwd` tiene una `x` en su lugar.
¿Cómo demuestras que confidencial es inaccesible?	`ls` o `cd` en confidencial con cualquiera de las tres cuentas devuelve Permission denied; solo `sudo ls` desde la cuenta de administración funciona.
¿Por qué no se usa root para trabajar?	Root no tiene comprobaciones de permisos; un error o un programa comprometido afecta a todo el sistema. Con sudo se eleva solo lo necesario y queda registro.